

Qul

$$\left. \begin{array}{l} 2^3 = 8 \\ 4^4 = 256 \end{array} \right\} \text{ Java: } \text{Math.pow}(2,3) = 8 \text{ --- } \log_2 n$$

A and n, find  $a^n$  using recursion. } - Recursion  
int

Sol

$$a^n \Rightarrow \underbrace{a * a * a * a \dots a}_{n \text{ times}}$$

```
int pow(a, n) {
    ans = 1
    for(i=1; i<=n; i++) {
        ans = ans * a;
    }
    return ans;
}
```

TC:  $O(n)$   
SC:  $O(1)$

Recursive

$a = 2 \quad b = 6$

$$\text{pow}(2,6) = 2 * \underbrace{2 * 2 * 2 * 2 * 2}_{2^5}$$

$$\text{pow}(2,6) = 2 * \text{pow}(2,5);$$

$$\text{pow}(a,n) = a * \text{pow}(a,n-1)$$

$$5^3 \Rightarrow 5 * \underbrace{5^2}_{\substack{\downarrow \\ 5 * \underbrace{5^1}_{\substack{\downarrow \\ 5 * \underbrace{5^0}_{\boxed{5 * 5^0}}}}}} \Rightarrow \underline{125 \text{ Ans}}$$

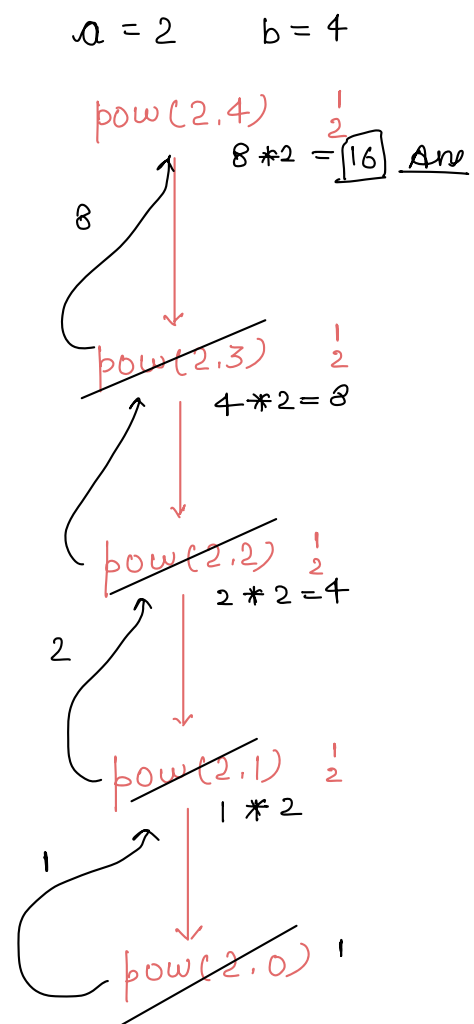
if  $n == 0$ , return 1

```

int pow(a, n) {
1  if (n == 0) {
        return 1;
    }
2  sa = pow(a, n-1);
3  return a * sa;
}

```

TC:  $O(n) \longrightarrow n * 1 = O(n)$   
 SC:  $O(n)$



## Optimisation

$$\text{pow}(a, n) = a * \text{pow}(a, n-1)$$

$$2^{10} = 2 * 2^9$$

$$\downarrow$$
$$2 * 2^8$$

$$\downarrow$$
$$2 * 2^7$$

$$\downarrow$$
$$2 * 2^6$$

$$\downarrow$$
$$2 * 2^5$$

⋮

$$2 * 2^0$$

$$2^{10} \Rightarrow 2^5 * 2^5$$

$$2^{11} \Rightarrow 2^5 * 2^5 * 2$$

n even ———

$$\text{pow}(a, n) = \text{pow}(a, \frac{n}{2}) * \text{pow}(a, \frac{n}{2})$$

n odd ———

$$\text{pow}(a, n) = \text{pow}(a, n/2) * \text{pow}(a, n/2) * a$$

Base case     $n == 0$   
                  return 1.

```
int pow(a, n) {
```

```
1 if (n == 0) {
    return 1;
}
```

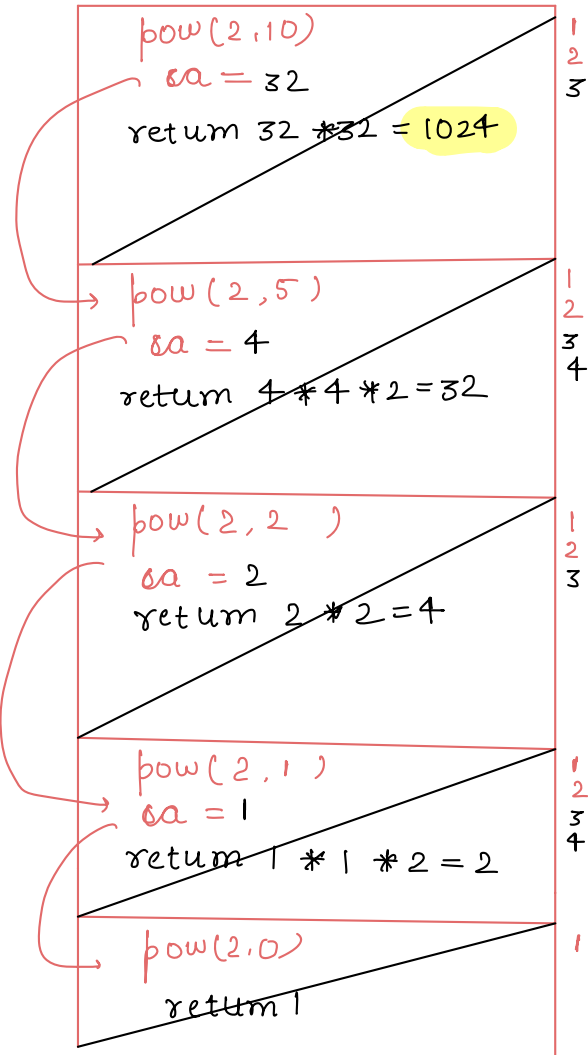
```
2 sa = pow(a, n/2);
```

```
3 if (n % 2 == 0) {
    return sa * sa;
}
```

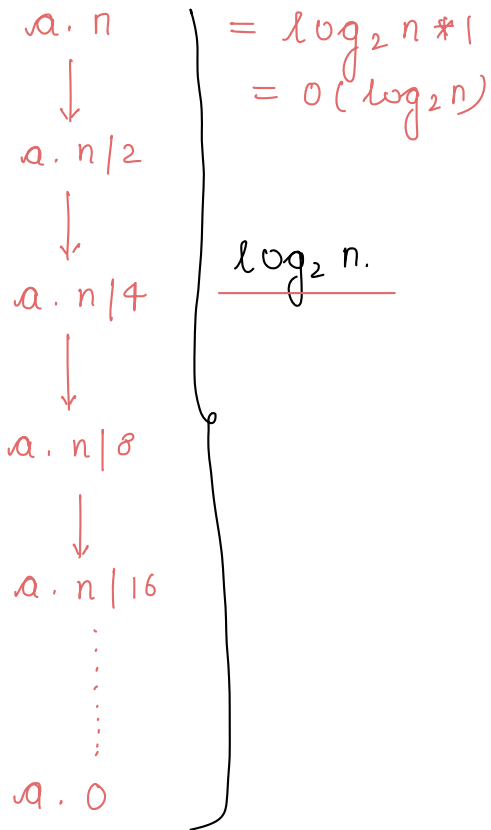
```
4 return sa * sa * a;
```

```
}
```

$a = 2, n = 10$



TC:



SC:  $O(\log_2 n)$

$a, 100$



$\log_2(100) = [6 \dots 7]$

Qu2     $A[]$  , print all el of array using recursion

$A =$

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 | 5 |

ans:    1   2   3   4   5

↓  
dynamic programming  
 $arr[]$

$arr[0-4] \Rightarrow$  1   2   3   4   5  
                                 $arr[1-4]$

$print(arr, \overset{idx}{\uparrow} 0, n-1)$   
↓

$print(A, \overset{0}{idx}, 4) \longrightarrow A[0] \quad \underline{print(A, 1, 4)}$

$\left\{ \begin{array}{l} A[idx] \\ print(arr, idx+1, n-1) \end{array} \right.$

↓  
 $A[1] \quad \underline{print(A, 2, 4)}$   
2

```
printArray (A[], idx) {  
    if( idx == A.length-1 ) {  
        print(A[idx]);  
    }  
    return;  
    print(A[idx]);  
    printArray(A, idx+1);  
}
```

↓  
3     $print(A, 3, 4)$   
↓  
4     $print(A, 4, 4)$   
↓

5     $print(A, 5, 4)$

TC:  $O(n)$   
SC:  $O(n)$

Qu

$A[]$ , target integer = B

find all indices at which B occur in array [ Rec ]

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| 4 | 5 | 3 | 1 | 5 | 4 | 5 |

target = 5

ans = { 1, 4, 6 }

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 1 | 1 |

target = 1

ans = { 0, 3, 4 }

```
List<Integer> findAllIndices ( A, target ) {  
    List<Integer> result = new ArrayList<>();  
    helper( A, 0, target, result );  
    return result;  
}  
  
void helper( A, idx, target, List<Integer> result ) {  
    if ( idx == A.length ) {  
        return; }  
    if ( A[idx] == target ) {  
        result.add( idx );  
    }  
    helper( A, idx + 1, target, result );  
}
```

TC:  $O(n)$   
SC:  $O(n)$

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| 4 | 5 | 3 | 1 | 5 | 4 | 5 |

target = 5

idx

helper(A, 0, 5, <>)  $\frac{1}{2}$   
 $\frac{3}{3}$

helper(A, 1, 5, <>)  $\frac{1}{2}$   
 $\frac{3}{3}$

helper(A, 2, 5, <1>)  $\frac{1}{2}$   
 $\frac{3}{3}$

helper(A, 3, 5, <1>)  $\frac{1}{2}$   
 $\frac{3}{3}$

helper(A, 4, 5, <1>)  $\frac{1}{2}$   
 $\frac{3}{4}$

helper(A, 5, 5, <1, 4>)  $\frac{1}{2}$   
 $\frac{3}{3}$

helper(A, 6, 5, <1, 4, 6>)  $\frac{1}{2}$   
 $\frac{3}{3}$

~~helper(A, 7, 5, <1, 4, 6>) |~~

```
List<Integer> findAllIndices(A, target) {
    List<Integer> result = new ArrayList<>();
    helper(A, 0, target, result);
    return result;
}
```

```
void helper(A, idx, target, List<Integer> result) {
    1 if (idx == A.length) {
        return;
    }
    2 if (A[idx] == target) {
        result.add(idx);
    }
    3 helper(A, idx + 1, target, result);
}
```

Base case.

Break: 10:22 - 10:32

Qu string, check if its a palindrome. [recursion]

radar = true.

area = false

m  
↑  
a l a y a l a  
m. ⇒  
pal(str, s, e)  
↓  
pal(str, s+1, e-1);

m  
↑  
a l a y x l a  
m.  
↑

a r e r f.  
↑  
false.

```
boolean palindrome(str) {  
    return palindrome(str, 0,  
                        n-1);  
}
```

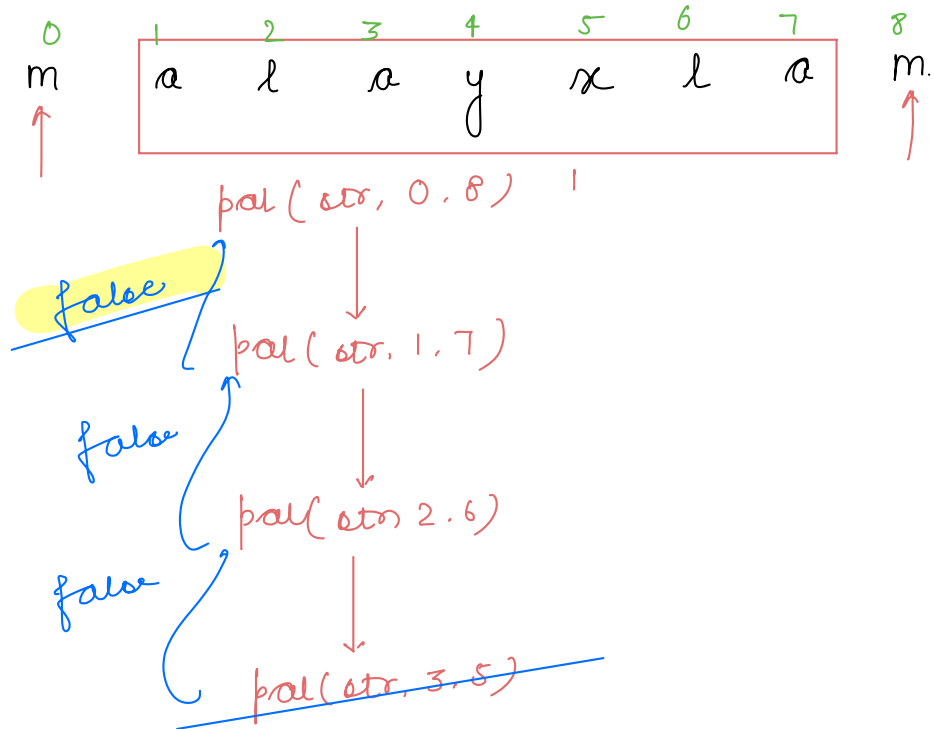
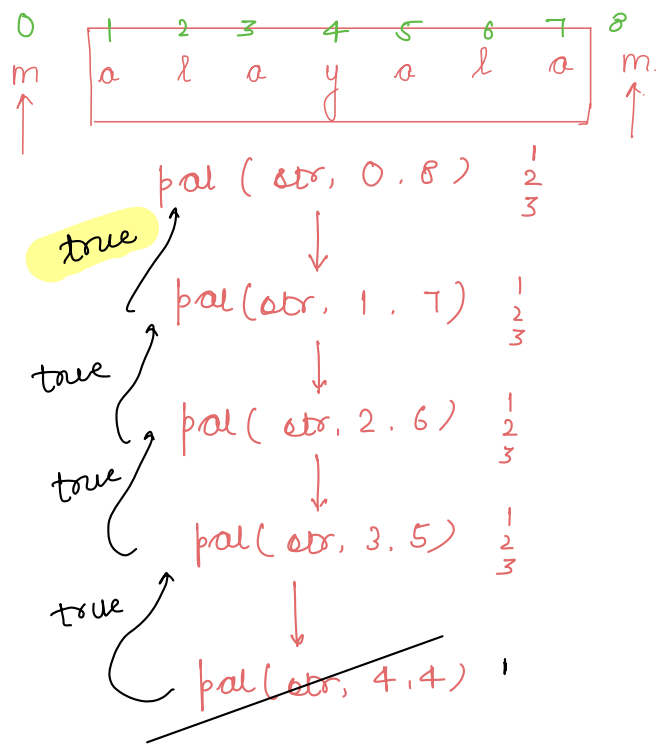
```
boolean palindrome(str, s, e) {  
    if (s >= e) {  
        return true;  
    }  
    if (str[s] != str[e]) {  
        return false;  
    }  
    return palindrome(str, s+1,  
                       e-1);  
}
```



```

boolean palindrome(str, s, e) {
    1 if (s >= e) {
        return true;
    }
    2 if (str[s] != str[e]) {
        return false;
    }
    3 return palindrome(str, s+1, e-1);
}

```



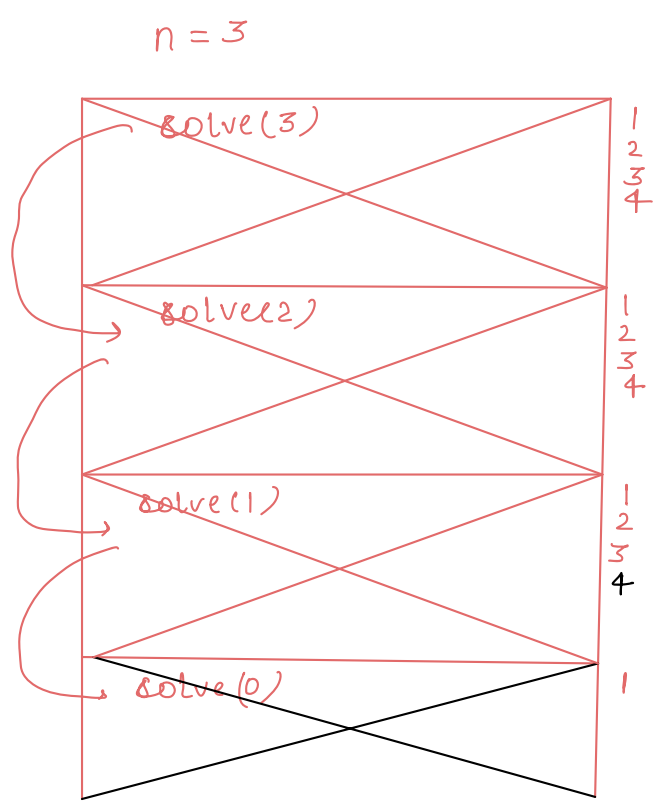
TC:  $O(n)$   
 SC:  $O(n)$

```

void solve(n) {
  1 if (n == 0)
    return
  2 print(n)
  3 solve(n-1)
  4 print(n)
}

```

3 2 1 1 2 3



```

solve(n) {
  1 if (n == 0)
    return
  2 print(n)
  3 solve(n-1)
}

```

stack overflow

$n = -3$

$\text{solve}(-3)$

$\text{solve}(-4)$

$\text{solve}(-5)$

$\text{solve}(-6)$

-7

-8

...

-3  
-4  
-5

Thankyou 😊

Doubt

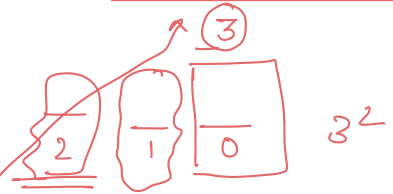
$3, 5 = 243$

$3 * 3 * 3 * 3 * 3$

Binary of 5 = 101

$n = 5$

Right shift



| <u>i</u> | <u>n</u> | <u>lab</u> | <u>a</u> | <u>ans</u>   |                      |
|----------|----------|------------|----------|--------------|----------------------|
| 1        | 5        | 1          | 3        | 1 * 3 = 3    |                      |
| 2        | 2 [10]   | 0          | 9        | 3            | <u>skip multiply</u> |
| 3        | 1        | 1          | 81       | 3 * 81 = 243 |                      |
| 4        | 0        |            | 243      | <u>Ans</u>   |                      |

if <sup>of n</sup> i<sup>th</sup> bit set

$ans = a^{[2^i]}$

Right shift

Recursion

$a = 3$

| 4   | 3   | 2  | 1 | 0 |
|-----|-----|----|---|---|
| ↑   | ↑   | ↑  | ↑ | ↑ |
| 729 | 243 | 81 | 9 |   |